Day 8 04/02/2026

Pre-increment / Post-increment

Pre-increment:

- The variable is incremented before its value is used in an expression. For example, if `x` is 5, then `++x` will first increment `x` to 6 and then use the value 6 in the expression.

e.g.

```
int x = 5;
int y = ++x; // x is incremented to 6, then y is assigned
the value of x (6)

cout << "x: " << x << ", y: " << y; // Output: x: 6, y: 6
```

Post-increment:

- The variable is incremented after its value is used in an expression. For example, if `x` is 5, then `x++` will first use the value 5 in the expression and then increment `x` to 6.

e.g.

```
int x = 5;
int y = x++; // y is assigned the value of x (5), then x is incremented to 6

cout << "x: " << x << ", y: " << y; // Output: x: 6, y: 5
```

- We can also overload the pre-increment and post-increment operators in C++ to define custom behavior for our classes. The pre-increment operator is typically defined as a member function that returns a reference to the object, while the post-increment operator is defined as a member function that takes an int parameter (to differentiate it from pre-increment) and returns a copy of the object before incrementing.

```
#include <iostream>
using namespace std;

class Complex{
    int real, imag;

public:
    //constructor overloading
    Complex() : real(0), imag(0){
    }
```

```
    Complex(int real, int imag) : real(real), imag(imag){
    }

    void display(){
        cout << real << " + " << imag << "i" << endl;
    }

    // member operator overloading
    Complex operator++(int){
        // temp object -> current obj state
        Complex temp = *this;   //10 + 20i

        real++;     //this->real;
        imag++;     // this->imag;

        return temp;        // return original state
    }

    // pre-increment
    Complex& operator++(){
        ++real;
        ++imag;
        return *this;
    }
};

int main(){
    Complex c1(10, 20);
    cout << "Original: ";
    c1.display();

    Complex c2 = c1++; // post-increment
    cout << "After post-increment: ";
    c1.display();
    cout << "Returned by post-increment: ";
    c2.display();

    Complex c3 = ++c1; // pre-increment
    cout << "After pre-increment: ";
    c1.display();
    cout << "Returned by pre-increment: ";
    c3.display();

    return 0;
}
```

> `operator++(int)` and `operator++()` are the overloaded post-increment and pre-increment operators for the Complex class, respectively.

The int parameter in `operator++(int)` is a dummy parameter used to differentiate it from the pre-increment operator. It is not used in the function body.

Stream operators (<<, >>)

Stream insertion operator (<<):

- The stream insertion operator (<<) is used to output data to the console or to a file. It is typically overloaded for user-defined types to allow for easy output of their contents.

Stream extraction operator (>>):

- The stream extraction operator (>>) is used to input data from the console or from a file. It is also typically overloaded for user-defined types to allow for easy input of their contents.

> The both operators are often implemented as non-member functions (friend functions) to allow for the left-hand operand to be of a different type (e.g., `ostream` or `istream`).

As the `cout` and `cin` objects are of type `ostream` and `istream` respectively, we cannot make the operator overloading functions as member functions of our class. Instead, we can declare them as friend functions to allow access to the private members of the class.

```
#include <iostream>
using namespace std;

class Complex{
    int real, imag;

public:

    //constructor overloading
    Complex() : real(0), imag(0){
    }

    Complex(int real, int imag) : real(real), imag(imag){
    }

    //  can be ignored as we are using friend functions for stream operators
    // void display(){
    //  cout << real << " + " << imag << "i" << endl;
    // }

    friend ostream& operator<<(ostream& os, Complex& other);
    friend istream& operator>>(istream& is, Complex& other);
};

ostream& operator<<(ostream& os, Complex& other){
    return os << other.real << " + " << other.imag << "i";
}

istream& operator>>(istream& is, Complex& other){
    return is >> other.real >> other.imag;
}

int main(){
```

```
    Complex c1(10, 20);
    cout << "Original: " << c1 << endl;

    Complex c2;
    cout << "Enter real and imaginary parts: ";
    cin >> c2;
    cout << "You entered: " << c2 << endl;

    return 0;
}
```

Assignment operator

- The assignment operator (=) is used to assign the value of one object to another. It can be overloaded to define custom behavior for assignment between objects of a class.
- When we overload the assignment operator, we typically need to handle self-assignment (where an object is assigned to itself) and ensure that we properly manage any dynamically allocated resources to avoid memory leaks.
- We need to overload the assignment operator as a member function of the class, and it should return a reference to the current object to allow for chained assignments.
- As the compiler provides a default assignment operator that performs a shallow copy, we need to overload it to perform a deep copy if our class contains pointers or dynamically allocated resources.

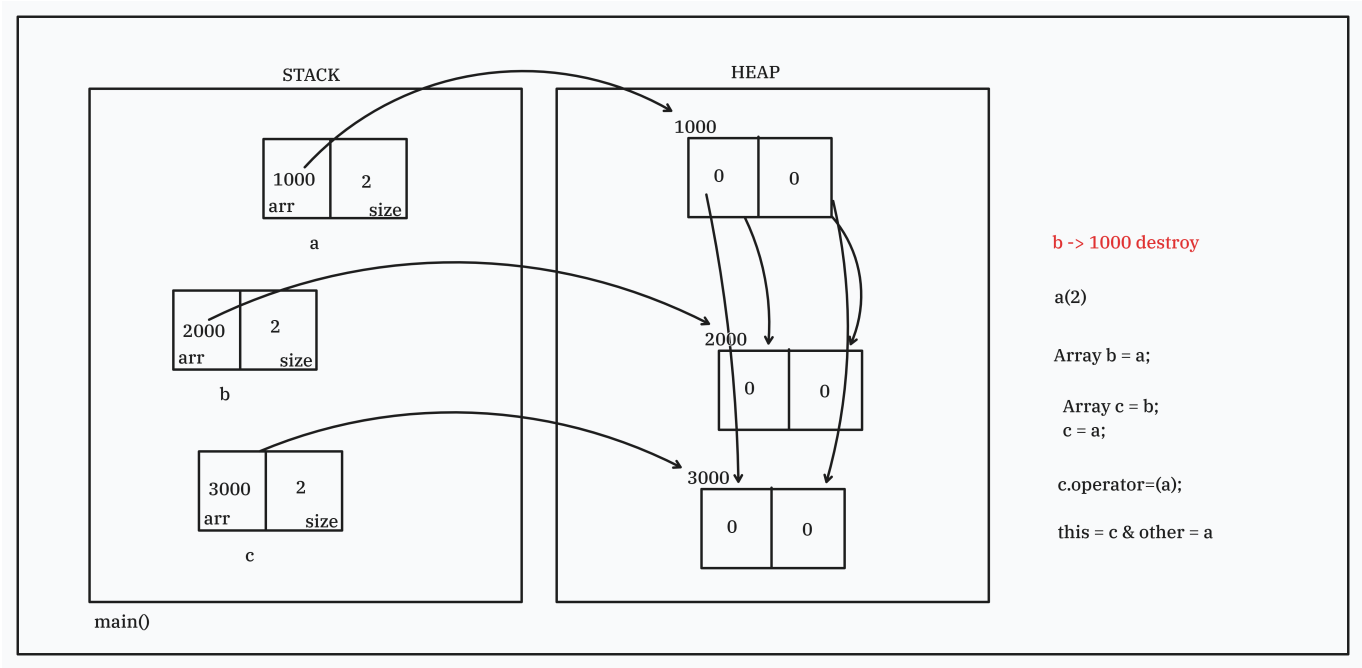
Subscript operator []

- The subscript operator ([]) is used to access elements of an array or a container class. It can be overloaded to allow for custom indexing behavior in user-defined classes.
- When overloading the subscript operator, we typically need to return a reference to the element being accessed to allow for both reading and writing of the element. We also need to handle out-of-bounds access to prevent undefined behavior.
- The subscript operator can be overloaded as a member function of the class, and it should take an index as a parameter and return a reference to the element at that index. We can also overload it to provide const access for read-only operations.

```
#include <iostream>
using namespace std;

class Array{
    int* arr;
    int size;

public:

    //default cons
    Array():arr(nullptr), size(0){
    }

    // para cons
    Array(int size) : size(size){
```

```
        this->arr = new int[size];

        for(int i=0;i<size;i++)
            arr[i] = 0;
    }

    // Deep Copy cons
    Array(const Array& other){
        this->size = other.size;
        arr = new int[size];

        for(int i=0;i<size;i++)
            this->arr[i] = other.arr[i];
    }

    // Assigment operator overloading   Array a; Array b = a; b = a; b.operator=
(a)
    Array& operator=(Array other){
        if(this == &other)      // a = a
            return *this;

        delete[] arr;       // older object ->

        this->size = other.size;
        arr = new int[size];
        for(int i=0;i<size;i++)
            this->arr[i] = other.arr[i];
        return *this;
    }

    int operator[](int index){
        if(index < 0 || index >=size)
            throw out_of_range("Out of Range...");
        return arr[index];  //int value
    }

    ~Array(){
        delete[] arr;
    }

};

int main(){

    Array a(2);     // size 2

    int num = a[0];

    Array b = a;        // copy cons        : a.arr -> 1000 b.arr -> 1000

    Array c;

    c = a;          // assignment : : a.arr -> 1000 b.arr -> 1000
```

```
    int val = b[1]; // index 2+1 = 3

    cout << val << endl;
    cout << num << endl;

    return 0;
}
```



Function overloading

- This we have already seen in the context of constructors, where we can define multiple constructors with different parameter lists to allow for different ways of initializing an object. However, function overloading is not limited to constructors; we can overload any function in C++ as long as the parameter lists are different.
- Function overloading is a feature in C++ that allows us to define multiple functions with the same name but different parameter lists. The compiler determines which function to call based on the number and types of arguments passed to the function.
- When we overload a function, we can provide different implementations for different types of data or different numbers of parameters, which can improve code readability and usability. The overloaded functions must differ in their parameter lists (either in the number of parameters or the types of parameters) to avoid ambiguity.

> Function signature matters in the context of function overloading. The function signature includes the function name and the parameter types (but not the return type). For example, `void func(int)` and `void func(double)` are considered different functions because they have different parameter types, while `void func(int)` and `int func(int)` would cause a compilation error due to having the same parameter list but different return types.

Name mangling:

- The process by which the compiler generates unique names for overloaded functions is called name mangling.
- When we overload functions, the compiler creates unique names for each function based on its signature (function name and parameter types).
- This allows the compiler to differentiate between the overloaded functions when they are called in the code.
- The exact name mangling scheme can vary between different compilers, but it typically involves encoding the function name along with the types of its parameters to create a unique identifier for each overloaded function.

We can check the name mangling by using the `nm` command on the compiled object file. For example, if we compile a C++ file with overloaded functions, we can run `nm` on the resulting object file to see the mangled names of the functions. This can help us understand how the compiler differentiates between overloaded functions.

Extern "C":

- The `extern "C"` declaration is used in C++ to indicate that a function should use C linkage instead of C++ linkage. This is often used when we want to call C functions from C++ code or when we want to prevent name mangling for certain functions.
- When we declare a function with `extern "C"`, the compiler will not mangle the function name, and it will use C-style linkage, which allows the function to be called from C code or from other languages that can interface with C.
- This is particularly useful when we want to create a C++ library that can be used by C programs or when we want to call C functions from our C++ code without worrying about name mangling.

```
#include <iostream>
using namespace std;

//return_type funcName(paramters){
//
//}

int add(int num1, int num2){
    return num1 + num2;
}

// function overloading
//int add(int num1, double num2){
//  return num1 + num2;
//}


//int add(int num1, double num2, int num3){
//  return num1 + num2 + num3;
//}

//parameters(int, int) (double, int)(int, double)

int add(double num1, int num2){
```

```
    return num1 + num2;
}

int add(double num2, int num1){
    return num1 + num2;
}

int main() {
    cout << add(10.5,20.5) << endl;

    return 0;
}
```

Abstraction concept

- Abstraction is hiding complex details and showing only the essential features of an object.
- Encapsulation is the process of bundling data and methods that operate on that data within a single unit (class) and restricting access to some of the object's components.

> `Encapsulation` = `Abstraction` + `Data Binding`

`Abstraction` is a concept and `Encapsulation` is a mechanism to achieve that concept.

- Abstraction is achieved in C++ via Access Specifiers (public, private, protected) and by using abstract classes and interfaces.
- Encapsulation is achieved by defining classes with private data members and public member functions to access and modify those data members.

Association

- Association is a relationship between two classes where one class uses or interacts with another class. It represents a "uses-a" relationship between objects of different classes.

- Association can be one-to-one, one-to-many, or many-to-many. For example, a `Student` class may have an association with a `Course` class, where a student can enroll in multiple courses and a course can have multiple students.

- Association is typically implemented in C++ by including an object of one class as a member variable in another class. This allows the classes to interact with each other while maintaining their own separate identities.

Aggregation

- Aggregation is a special form of association that represents a "has-a" relationship between classes. It indicates that one class (the whole) contains or is composed of other classes (the parts), but the parts can exist independently of the whole.
- For example, a `University` class may have an aggregation relationship with a `Faculty` class, where a university has multiple faculties, but the faculties can exist independently of the university.

Composition

- Composition is a stronger form of aggregation where the parts cannot exist independently of the whole. In composition, the lifetime of the parts is managed by the whole, meaning that when the whole is destroyed, the parts are also destroyed. For example, a `Human` class may have a composition relationship with a `Heart` class, where the heart cannot exist without the human.

```
#include <iostream>
using namespace std;


// encapsulation
//class Student{
//  int data_member;
//public:
//  member_functions
//};

// Association example
class Printer{
public:
    void printJob(){
        cout << "Printing..." << endl;
    }
};

class Computer{

public:
    void print(Printer* p){
        p->printJob();
    }
};

// Aggregation example
class Faculty{
    string name;
public:

    Faculty(string name): name(name){

    }

    void display(){
        cout << name << endl;
    }

};

class University{
 Faculty* faculty;

public:
```

```
    University(Faculty* faculty): faculty(faculty){

    }

    void facultyData(){
        faculty->display();
    }

};

// Composition example
class Heart{
public:
    void beats(){
        cout << "beating..." << endl;
    }
};

class Human{
public:
    Heart heart;        // composition -> object
};

int main() {
    Computer c;
    Printer p;

    c.print(&p);        // association

    Faculty f("shil");

    University u(&f);   // aggregation

    u.facultyData();

    Human h;

    h.heart.beats();        // composition

    return 0;
}
```

Relationship modeling

- In object-oriented design, we can model relationships between classes using association, aggregation, and composition to represent how objects of different classes interact with each other. This helps in designing a system that is modular, maintainable, and scalable.
- By understanding and applying these concepts, we can create a well-structured object-oriented design that accurately represents the real-world relationships between entities in our system.

The Rule of Relationships:

- If the relationship is "uses-a", then it is an association.
- If the relationship is "has-a" and the parts can exist independently, then it is an aggregation.
- If the relationship is "has-a" and the parts cannot exist independently, then it is a composition.
- If the relationship is "is-a", then it is an inheritance.

> Aggregation: Weak Has-a relationship | Loose coupling Composition: Strong Has-a relationship | Tight coupling

Inheritance concept

- Inheritance is a fundamental object-oriented programming concept that allows a new class (called a derived class or child class) to inherit states and behaviors (data members and member functions) from an existing class (called a base class or parent class).
- This promotes code reusability and establishes a natural hierarchical relationship between classes.
- When you have to define `Is-a` relationship between classes, inheritance is the way to go.
- For example, if we have a `Animal` class and a `Dog` class, we can say that a dog is an animal, so we can use inheritance to create the `Dog` class as a derived class of the `Animal` class.

Modes of inheritance

- In C++, there are three modes of inheritance: public, protected, and private. The mode of inheritance determines the access level of the inherited members in the derived class.
- Public inheritance: In public inheritance, the public members of the base class become public members of the derived class, and the protected members of the base class become protected members of the derived class. The private members of the base class are not accessible in the derived class.
- Protected inheritance: In protected inheritance, the public and protected members of the base class become protected members of the derived class. The private members of the base class are not accessible in the derived class.
- Private inheritance: In private inheritance, the public and protected members of the base class become private members of the derived class. The private members of the base class are not accessible in the derived class.

> `Public` Mode of inheritance is the most commonly used mode, as it allows for a clear "is-a" relationship between the base and derived classes. `Protected` and `Private` modes of inheritance are less commonly used and are typically used in specific scenarios where we want to restrict access to the inherited members in the derived class.

It is recommended to use composition over inheritance whenever possible, as it promotes better encapsulation and reduces coupling between classes.

Inheritance should be used when there is a clear "is-a" relationship between the base and derived classes, while composition should be used when there is a "has-a" relationship between classes.

Also it is recommended to prefer public inheritance over protected or private inheritance, as it maintains a clear and intuitive relationship between the base and derived classes. Private and protected inheritance can lead to confusion and should be used with caution (better to use composition instead).

Types of inheritance

Single inheritance:

In single inheritance, a derived class inherits from a single base class.

For example, a `Dog` class can inherit from an `Animal` class.

Multiple inheritance:

In multiple inheritance, a derived class inherits from more than one base class.

For example, a `Smart Fan` class can inherit from both a `Fan` class and a `SmartDevice` class.

Multilevel inheritance:

In multilevel inheritance, a derived class inherits from a base class, and then another class inherits from that derived class.

For example, a `Poodle` class can inherit from a `Dog` class, which in turn inherits from an `Animal` class.

Hierarchical inheritance:

In hierarchical inheritance, multiple derived classes inherit from a single base class. For example, a `Car` class can be a base class for `Sedan`, `SUV`, and `Truck` classes.

Hybrid inheritance:

Hybrid inheritance is a combination of two or more types of inheritance. It can involve multiple inheritance, multilevel inheritance, or hierarchical inheritance in various combinations.

For example, a `Person` class can be a base class for both `Teacher` and `Student` classes (hierarchical inheritance), and then an `TeachingAssistant` class can inherit from both `Teacher` and `Student` classes (multiple inheritance).

```
#include <iostream>
using namespace std;

class Animal{
protected:
    int legs;
public:

    void eat(){
        cout << "Animal eating.." << endl;
    }

    void makeSound(){
        cout << "Animal Make Sound.." << endl;
    }

    void walk(){
        cout << "Animal Walking.." << endl;
    }
```

```
};  // Base Class : Parent Class : Generalization


// Mode of Inheritance-  public : protected : private
class Human : public Animal{
public:
    void think(){
        cout << "Human thinking.." << endl;
    }

    void code(){
        cout << "Human coding.." << endl;
    }

    void showLegs(){
        cout << legs << endl;
    }
};  //Derived Class : Child Class : Specialization


// Types of Inheritance
// Single       : One Base class -> One Derived class
// Multiple     : Multiple Base classes -> One Derived class
// Multilevel   : One Base class -> One Derived class -> Another Derived Class
// Heirarchical : One Base class -> Multiple Derived classes
// Hybrid       : Combination that cause Diamond shape


// Single Inheritance
class A{
public:
    int x;  //4
};


class B : virtual public A{

};

// Multilevel
class C : public B{

};

// Heirarchical
class D : virtual public A{

};


// Multiple - Hybrid
class E : public B, public D{

};
```

```
//   A
//  / \
//  B  D
//  \ /
//   E

// Diamond problem


int main(){

    E e;

    cout << sizeof(e) << endl;
    cout << e.x << endl;        //Diamond Problem

    return 0;
}

int main1() {

    Human h;        // Derived class

    h.eat();        // Base class
    h.think();  // Derived class

    return 0;
}
```